



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
شبکه‌های عصبی و یادگیری عمیق
تمرین دوم

نام و نام خانوادگی	امیرحسین راحتی
شماره دانشجویی	۸۱۰۱۰۴۳۰۳
تاریخ ارسال گزارش	۱۵ اردیبهشت ۱۴۰۵

فهرست

3.....	سؤال اول
3.....	بخش اول: سوالات تئوری
3.....	زیربخش اول: تابع زیان طبقه بند کلاسیک
4.....	زیربخش دوم: توابع زیان و فاصله بین کلاسی
4.....	زیربخش سوم: ارتباط فاصله زاویه ای به تشخیص چهره
5.....	زیربخش چهارم: تابع زیان IAM
6.....	زیربخش پنجم: توابع زیان دیگر
7.....	بخش دوم: سوالات پیاده سازی
7.....	زیربخش اول: آماده سازی داده
8.....	زیربخش دوم: مدل پایه CNN با SOFTMAX
12.....	زیربخش سوم: پیاده سازی ANGULAR MARGIN LOSS
15.....	زیربخش چهارم: مقایسه نهایی، تحلیل و ارتباط با مقاله IAM
16.....	مراجع

سؤال اول

بخش اول: سوالات تئوری

زیربخش اول: تابع زبان طبقه بند کلاسیک

(الف)

$$\text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}$$

این تابع ورودی‌ها را به یک توزیع احتمال تبدیل می‌کند به طوری که جمع همه خروجی‌ها برابر یک شود. در طبقه‌بندی، نمونه به کلاسی نسبت داده می‌شود که بیشترین مقدار احتمال را داشته باشد.

(ب)

چون Softmax فقط به این توجه دارد که خروجی مربوط به کلاس درست بیشتر از بقیه باشد، ولی هیچ مکانیزمی برای دور کردن مراکز کلاس‌ها از یکدیگر یا جمع کردن نمونه‌های یک کلاس ندارد. بنابراین ممکن است مرزهای تصمیم بین کلاس‌ها بسیار نزدیک باشد و ویژگی‌های کلاس‌های مختلف تفکیک پذیری کافی نداشته باشند.

(ج)

در این روش ابتدا بردار ویژگی x و وزن‌های کلاس w را نرمالیزه می‌کنیم تا نرم آنها برابر یک شود. سپس حاصل ضرب داخلی به کسینوس زاویه بین دو بردار تبدیل می‌شود. برای جلوگیری از کوچک شدن گرادیان‌ها، این مقدار کسینوس را در یک ضریب مقیاس s ضرب می‌کنیم. تابع زبان به شکل زیر تعریف می‌شود:

$$L_{scale} = -\frac{1}{N} \sum_{i=1}^N \log_2 \frac{e^{s \cos \theta_{i,y_i}}}{\sum_{j=1}^C e^{s \cos \theta_{i,j}}}$$

(د)

این تابع برای رفع مشکل همگرایی معرفی می‌شود، چون بعد از نرمالیزه کردن، مقدار ورودی لایه آخر بین -1 و 1 محدود می‌شود که باعث کوچک شدن گرادیان و سخت شدن آموزش می‌شود. ضریب s این مشکل را تا حدی حل می‌کند. اما تابع scaled به تنهایی نمی‌تواند فاصله درون کلاس‌ها را کم کند یا فاصله بین کلاس‌ها را زیاد کند و فقط بر اساس زوایای موجود بین ویژگی و وزن‌ها عمل می‌کند.

زیربخش دوم: توابع زبان و فاصله بین کلاسی

(الف)

inter-class variance به میزان پراکندگی یا فاصله بین نمونه‌های کلاس‌های مختلف گفته می‌شود. هرچه این مقدار بزرگتر باشد، ویژگی‌های مربوط به کلاس‌های متفاوت از هم دورتر هستند و تشخیص آنها از یکدیگر آسانتر است. در مقابل، intra-class distance به فاصله نمونه‌های درون یک کلاس نسبت به مرکز آن کلاس اشاره دارد.

(ب)

در تشخیص چهره، هدف این است که دو تصویر از یک شخص به درستی مشابه و دو تصویر از دو شخص متفاوت به درستی غیرمشابه تشخیص داده شوند. اگر inter-class variance کم باشد، ویژگی‌های افراد مختلف به هم نزدیک شده و شبکه ممکن است آنها را با یکدیگر اشتباه بگیرد. افزایش این فاصله باعث می‌شود مرزهای تصمیم بین کلاس‌ها واضحتر شود و خطای شناسایی کاهش پیدا کند.

(ج)

در روش‌های مبتنی بر فاصله اقلیدسی، بزرگی بردار ویژگی نیز در محاسبه فاصله تاثیر دارد. این موضوع مشکل ساز است چون تصاویری با کیفیت پایین یا نویز بالا ممکن است نرم کوچکتري داشته باشند و به اشتباه به کلاس‌های دیگر نزدیک محاسبه شوند. اما در فاصله زاویه‌ای، ابتدا بردارهای ویژگی و وزن‌ها نرمالیزه می‌شوند تا روی یک کره واحد قرار بگیرند. در این حالت فقط زاویه بین آنها اهمیت دارد و اثر بزرگی بردار حذف می‌شود. این ویژگی باعث می‌شود مدل فقط به جهت ویژگی توجه کند و نسبت به تغییرات شدت روشنایی یا کیفیت تصویر مقاومتر باشد.

زیربخش سوم: ارتباط فاصله زاویه ای به تشخیص چهره

اگر ویژگی‌ها و وزن‌ها را نرمالیزه کنیم، یعنی نرم هر دو بردار را برابر یک قرار دهیم، آنگاه رابطه زیر برقرار خواهد شد:

$$\cos(\theta) = \frac{w^T x}{|w||x|}$$

یعنی مقدار ضرب داخلی دو بردار نرمال سازی شده دقیقاً برابر کسینوس زاویه بین آنها خواهد بود.

$$\cos(\theta) = w^T x$$

وقتی بردارها روی کره واحد قرار دارند، دیگر بزرگی آنها تأثیری در محاسبه شباهت ندارد و فقط زاویه بین آنها مهم میشود. هرچه زاویه بین ویژگی یک نمونه و وزن کلاس مربوطه کوچکتر باشد، مقدار کسینوس به یک نزدیکتر است و در نتیجه شباهت بیشتری وجود دارد. از طرف دیگر، اگر زاویه بزرگ باشد، کسینوس کوچک یا منفی میشود و نشان میدهد که نمونه به آن کلاس تعلق ندارد.

این نمایش برای تشخیص چهره بسیار مناسب است چون تأثیر عواملی مثل کیفیت تصویر، نورپردازی و کنتراست را تا حد زیادی حذف میکند. در روشهای مبتنی بر فاصله اقلیدسی، یک تصویر تیره ممکن است نرم کوچکی داشته باشد و به اشتباه به کلاس دیگری نزدیک محاسبه شود، اما در فضای زاویهای فقط جهت بردار ویژگی اهمیت دارد و این مشکل رفع میشود. همچنین آموزش شبکه در این فضا پایدارتر است و مدل یاد میگیرد که ویژگیهای چهره را بر اساس زاویه و نه بزرگی تفکیک کند.

زیربخش چهارم: تابع زیان IAM

(الف)

مقاله دو تابع زیان اصلی دارد. اولی یک تابع پایه مثل Softmax مقیاس دار است و دومی تابع جدید IAM به عنوان یک جمله منظم ساز. تابع IAM به شکل زیر تعریف میشود:

$$L_{IAM} = \frac{1}{N} \sum_{i=1}^N \log_2 \frac{\sum_{j \neq y_i} e^{s \cos \theta_{i,j}}}{\sum_{j=1}^C e^{s \cos \theta_{i,j}}}$$

و تابع کلی به صورت $L = L_{base} + \beta L_{IAM}$ است. این تابع باعث میشود زوایای بین کلاس هایی که به هم نزدیک هستند بیشتر جریمه شوند و در نتیجه فاصله زاویه ای بین کلاس ها افزایش پیدا کند.

(ب)

وقتی زاویه بین کلاس ها کوچک باشد، مقدار $\cos \theta_{i,j}$ برای کلاس های اشتباه بزرگ میشود. این باعث میشود numerator در تابع IAM بزرگ شده و مقدار کل تابع افزایش یابد. در نتیجه گرادیان نسبت به وزن کلاس های اشتباه نیز بزرگتر خواهد شد. این گرادیان بزرگتر به وزن ها فرمان میدهد که از ویژگی نمونه دورتر شوند و فاصله زاویه ای را افزایش دهند. به عبارت دیگر، هرچه دو کلاس به هم نزدیکتر باشند، IAM loss نیروی بیشتری برای جدا کردن آنها اعمال میکند.

(ج)

الگوریتم آموزش در مقاله به صورت کدگونه ارائه شده است. مراحل اصلی به این شکل است:

۱. ابتدا یک مینی بچ از داده های آموزش برداشته میشود.
۲. ویژگی های استخراج شده و وزن های لایه آخر هر دو نرمالیزه میشوند.
۳. مقدار تابع زیان پایه و تابع IAM محاسبه شده و با هم جمع میشوند.
۴. گرادیان نسبت به ویژگی ها و نسبت به وزن ها محاسبه می شود.
۵. پارامترهای شبکه و وزن های لایه آخر با استفاده از بهینه سازی مثل RMSPROP به روز رسانی میشوند.
۶. این مراحل تا تعداد تکرار مشخص شده ادامه پیدا میکند.

در این فرآیند، جمله IAM مانند یک نیروی اضافی عمل میکند که شبکه را مجبور میکند کلاس ها را از هم دور نگه دارد.

(د)

هایپریامتر اصلی در این روش β است که وزن جمله IAM را نسبت به تابع پایه مشخص میکند. نویسندگان از دیدگاه پس انتشار نشان داده‌اند که مقدار β باید کمتر از ۱ باشد. اگر $\beta=0$ باشد، عملاً جمله IAM حذف شده و فقط تابع پایه استفاده میشود. اگر β بین ۰ و ۱ باشد، جمله IAM به افزایش فاصله بین کلاس‌ها و کاهش فاصله درون کلاس‌ها کمک میکند و دقت بهبود مییابد. اگر $\beta=1$ باشد، وزن کلاس به مرکز کلاس تبدیل میشود. اما اگر $\beta>1$ باشد، رفتار معکوس رخ میدهد یعنی گرادیان برای نمونه‌هایی که فاصله درون کلاس کوچکی دارند بیشتر میشود که خلاف هدف است، همچنین برای زوایای بین کلاس کوچک، گرادیان کاهش مییابد که باعث کاهش فاصله بین کلاس‌ها میشود. بنابراین β باید حتماً کمتر از ۱ انتخاب شود و معمولاً مقادیری مثل ۰.۵، ۰.۱، ۰.۸ و ۰.۹ در آزمایشها نتایج خوبی داده‌اند.

زیربخش پنجم: توابع زیان دیگر

(الف)

تابع SphereFace که زاویه بین ویژگی نمونه و وزن کلاس صحیح را در یک ضرب m ضرب میکند. فرم آن به صورت زیر است:

$$L_{sphere} = -\frac{1}{N} \sum_{i=1}^N \log_2 \frac{e^{|x_i| \cos(m\theta_{i,y_i})}}{e^{|x_i| \cos(m\theta_{i,y_i})} + \sum_{j \neq y_i} e^{|x_i| \cos \theta_{i,j}}}$$

این کار باعث میشود یک حاشیه زاویه‌ای بین کلاس‌ها ایجاد شود. به این معنی که برای طبقه‌بندی یک نمونه به کلاس صحیح، زاویه آن باید بسیار کوچکتر از حالت عادی باشد. در نتیجه فاصله درون کلاس کم و فاصله بین کلاس زیاد میشود.

(ب)

تابع CosFace یا AM-Softmax به جای ضرب کردن در زاویه، یک مقدار ثابت m را از کسینوس زاویه کلاس صحیح کم میکند:

$$L_{cos} = -\frac{1}{N} \sum_{i=1}^N \log_2 \frac{e^{s(\cos \theta_{i,y_i} - m)}}{e^{s(\cos \theta_{i,y_i} - m)} + \sum_{j \neq y_i} e^{s \cos \theta_{i,j}}}$$

این روش نیز یک حاشیه ایجاد میکند با این تفاوت که روی مقدار کسینوس اعمال میشود و به راحتی میتوان آن را بدون نیاز به Softmax کمکی آموزش داد.

(ج)

تابع ArcFace مشابه CosFace است با این تفاوت که حاشیه m را مستقیماً به خود زاویه اضافه میکند نه به کسینوس آن:

$$L_{arc} = -\frac{1}{N} \sum_{i=1}^N \log_2 \frac{e^{s \cos(\theta_{i,y_i} + m)}}{e^{s \cos(\theta_{i,y_i} + m)} + \sum_{j \neq y_i} e^{s \cos \theta_{i,j}}}$$

این اصلاح باعث میشود حاشیه ایجاد شده دقیقاً روی زاویه اعمال شود و تفسیر هندسی واضحتری داشته باشد. در عمل ArcFace عملکرد بهتری نسبت به CosFace در بسیاری از مجموعه‌های داده نشان داده است.

(د)

تفاوت اصلی در این است که SphereFace و NormFace و حتی CosFace و ArcFace همگی از یک حاشیه ثابت برای همه کلاس‌ها استفاده میکنند، در حالی که IAM loss یک جمله منظم‌ساز است که به صورت وفقی عمل میکند. یعنی اگر دو کلاس به هم نزدیک باشند، IAM loss جریمه بیشتری اعمال میکند و اگر از هم دور باشند، تأثیر کمتری دارد. همچنین IAM loss به تنهایی یک تابع زیان کامل نیست بلکه میتواند به عنوان یک افزونه به روش‌های دیگر اضافه شود. در مقابل SphereFace برای همگرا شدن نیاز به Softmax کمکی دارد، اما IAM loss به راحتی با هر بهینه‌سازی قابل آموزش است.

(ج)

مقاله IAM loss را با هدف رفع محدودیت حاشیه ثابت در روش‌های قبلی معرفی کرده است. در روش‌های قبلی مثل SphereFace، CosFace و ArcFace، مقدار حاشیه برای همه کلاس‌ها و در همه مراحل آموزش یکسان است. اما در عمل، بعضی از کلاس‌ها ذاتاً به هم نزدیکتر هستند و نیاز به حاشیه بزرگتری دارند، در حالی که بعضی دیگر از هم دور هستند و حاشیه کوچک کافی است IAM loss. این مشکل را حل میکند چون به صورت تطبیقی هر جا که زاویه بین کلاس‌ها کوچک باشد، جریمه بیشتری اعمال میکند و باعث میشود شبکه یاد بگیرد که کلاس‌های نزدیک را از هم دور کند.

بخش دوم: سوالات پیاده سازی

زیربخش اول: آماده سازی داده

در این قسمت ابتدا ۱۰ میانگین را از توزیع یونیفرم ایجاد کردیم و سپس برای هرکدام تعداد ۲۰۰۰ نمونه از توزیع نرمال تخمین زدیم که عکس‌های ۱۴در۱۴ خواسته شده را ایجاد کردند. در این قسمت برای نتایج بهتر واریانس داده‌ها را افزایش دادیم. چون در غیراینصورت نتایج شبکه‌ها دقت ۱۰۰ درصدی میگرفتند.

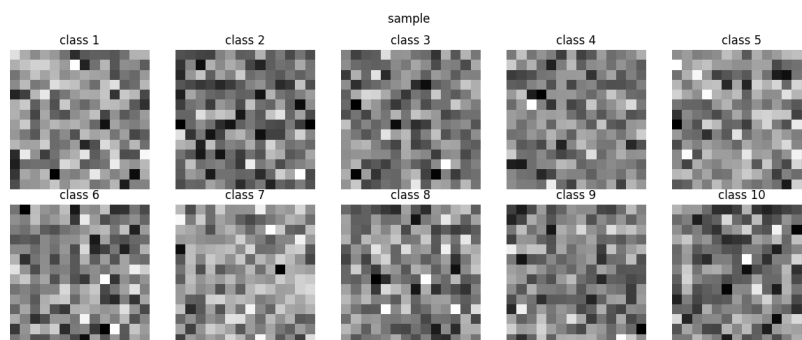
```
means = np.random.uniform(low=-5, high=5, size=(num_classes, feature_dim))

for class_id in range(num_classes):
    class_mean = means[class_id]
    cov = np.eye(feature_dim) * variance
    class_samples = np.random.multivariate_normal(class_mean, cov, samples_per_class)
    class_samples = class_samples.reshape(samples_per_class, img_size, img_size)

    all_images.append(class_samples)
    all_labels.append(np.full(samples_per_class, class_id + 1))
```

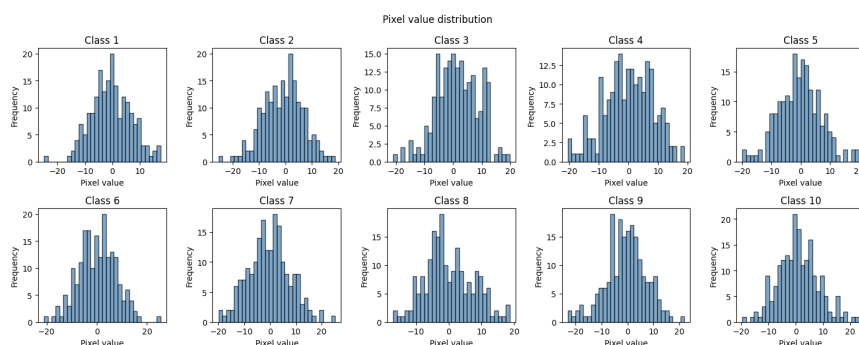
شکل ۱- ایجاد نمونه‌ها

چند نمونه از داده‌های کلاس‌ها به صورت زیر است:

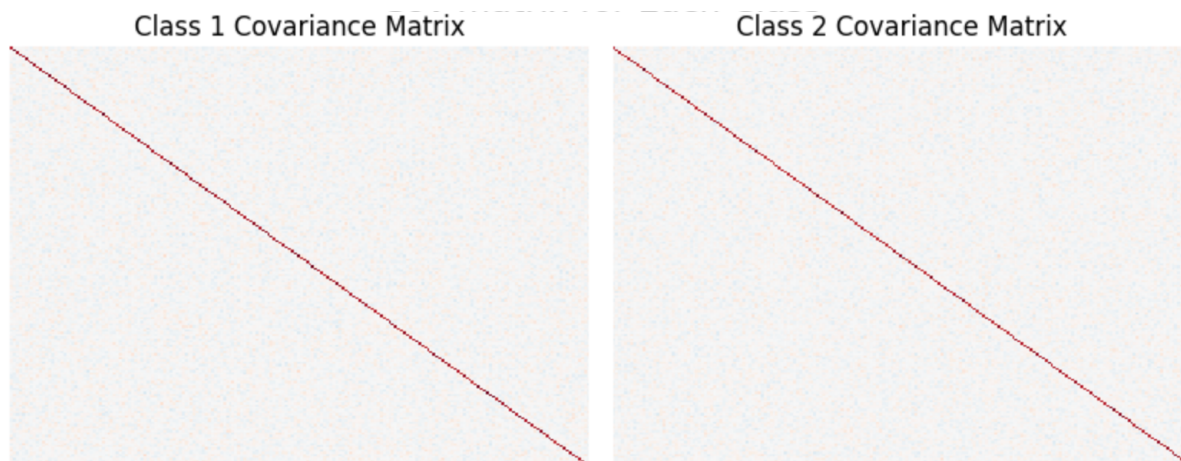


شکل 2- چند نمونه از داده های ایجاد شده

همچنین هیستوگرام توزیع داده ها و ماتریس کواریانس دو کلاس به عنوان نمونه به صورت زیر است



شکل 3- هیستوگرام توزیع داده ها



شکل 4 - ماتریس کواریانس داده ها

زیربخش دوم: مدل پایه CNN با SOFTMAX

الف) در این بخش شبکه ای تقریباً یکسان با شبکه پیاده سازی شده در مقاله پیاده سازی شد. با این فرق که به جای ورودی ۲۸ در ۲۸ داده های MNIST از ورودی های ۱۴ در ۱۴ تولید شده استفاده کنیم. ساختار این شبکه به صورت زیر است:

```

class MNISTNetwork(nn.Module):
    def __init__(self, num_classes=10):
        super(MNISTNetwork, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 1 * 1, 256),
            nn.ReLU(),
            nn.Linear(256, num_classes)
        )

```

شکل 5- شبکه *MNIST* مقاله

همانطور که مشخص است شبکه دارای دولایه کانولوشنی پشت سر هم ، یک لایه نرمالیزاسیون RELU و maxpool است که این ساختار ۳ بار تکرار میشود. تعداد پارامتر ها و ساختار شبکه به صورت زیر است:

```

Total parameters: 322,026
network structure: MNISTNetwork(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=128, out_features=256, bias=True)
    (2): ReLU()
    (3): Linear(in_features=256, out_features=10, bias=True)
  )
)

```

شکل 6- ساختار شبکه *MNIST* و تعداد پارامتر ها

(ب)

در این بخش، با ۳۰ ایپاک و استفاده از بهینه ساز adam و تابع هزینه crossEntropy شبکه را آموزش می‌دهیم:

```
def train_model(model, train_loader, test_loader, epochs=30, lr=0.001):
    device = torch.device('mps' if torch.cuda.is_available() else 'cpu')
    model = model.to(device)

    weight_decay = lr * 0.01
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)

    train_losses, test_losses = [], []
    train_accs, test_accs = [], []

    for epoch in range(epochs):
        model.train()
        train_loss = 0
        correct_train = 0
        total_train = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

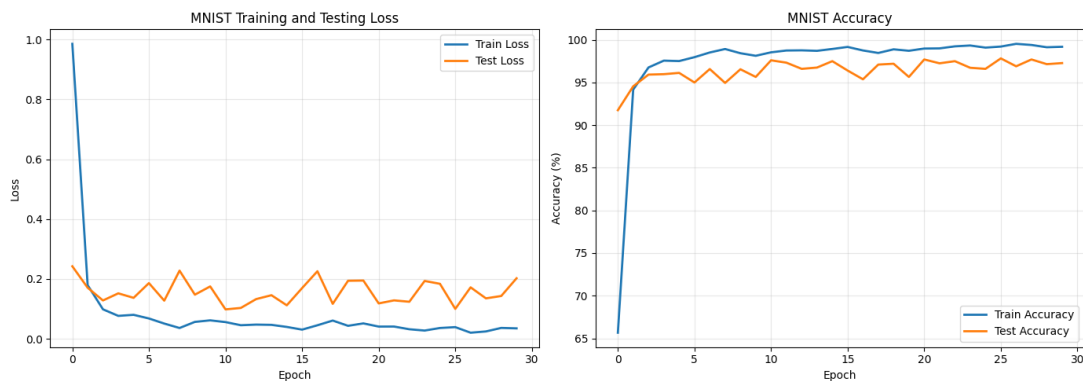
            train_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total_train += 1
            correct_train += predicted == labels
```

شکل 7 - بخشی از کد شبکه آموزش

نتایج آموزش شبکه به صورت زیر است که دقت ۹۱ درصدی روی داده تست را نشان میدهد. همچنین نمودار دقت و خطا هم به شکل زیر است.

Epoch	5/30	Train Loss:	0.0799	Train Acc:	97.51%	Test Loss:	0.1367	Test Acc:	96.12%
Epoch	10/30	Train Loss:	0.0614	Train Acc:	98.13%	Test Loss:	0.1748	Test Acc:	95.65%
Epoch	15/30	Train Loss:	0.0395	Train Acc:	98.93%	Test Loss:	0.1117	Test Acc:	97.50%
Epoch	20/30	Train Loss:	0.0513	Train Acc:	98.72%	Test Loss:	0.1944	Test Acc:	95.65%
Epoch	25/30	Train Loss:	0.0359	Train Acc:	99.09%	Test Loss:	0.1835	Test Acc:	96.60%
Epoch	30/30	Train Loss:	0.0347	Train Acc:	99.19%	Test Loss:	0.2020	Test Acc:	97.28%

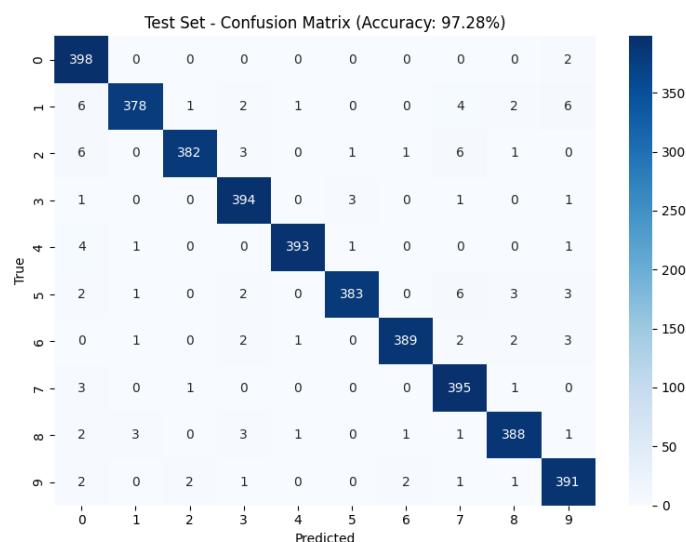
شکل 8 - نتایج آموزش روی داده های تست و ترین



شکل 9 - نمودار دقت و خطای داده های تست و ترین

کاهش اندازه batch size باعث بیشتر شدن فاصله دقت train و test میشود.

ج)



شکل 10 - confusion matrix برای داده های تست

در داده ها اورفیت مشاهده نمیشود. زیرا تفاوت دقت مدل روی داده های train و test تقریباً 3 درصد است که با توجه به ماهیت داده ها طبیعی بنظر میاید.

همچنین در شکل زیر کلاس هایی که بیشتر با هم اشتباه گرفته شده اند مشاهده میشود. این کلاس ها دارای میانگین های نزدیک به هم هستند که باعث ایجاد خطا در آنها میشود:

```
Class 0 often confused with: [9]
Class 1 often confused with: [0, 9, 7]
Class 2 often confused with: [0, 7, 3]
Class 3 often confused with: [5, 0, 7]
Class 4 often confused with: [0, 1, 5]
Class 5 often confused with: [7, 8, 9]
Class 6 often confused with: [9, 3, 7]
Class 7 often confused with: [0, 2, 8]
Class 8 often confused with: [1, 3, 0]
Class 9 often confused with: [0, 2, 6]
```

شکل 11 - میزان اشتباه بین کلاس ها

در تحلیل pca داده ها و نتایج با اعمال تابع softmax، تقریباً ۵۷ درصد واریانس توسط دو فیچر اول توضیح داده میشوند. اما در نمودار pca جداسازی نشان داده میشود.

```
Variance explained by PC1: 29.79%
Variance explained by PC2: 27.38%
Total variance explained: 57.17%
```

شکل 12 - تحلیل PCA طبقه بند



شکل 13 - نمودار داده ها بر اساس دو فیچر با بالاترین واریانس

همچنین در تصاویر زیر نتیجه محاسبه مرکز داده ها و واریانس درون کلاسی و بین کلاسی نشان داده شده است:

```
Class 0: center = (4.70, -47.30), intra-class spread = 25.08
Class 1: center = (-13.00, 19.81), intra-class spread = 11.97
Class 2: center = (-30.87, 34.77), intra-class spread = 17.58
Class 3: center = (-34.98, 16.54), intra-class spread = 20.65
Class 4: center = (79.87, -22.30), intra-class spread = 36.52
Class 5: center = (-4.21, -7.97), intra-class spread = 17.81
Class 6: center = (11.78, 47.24), intra-class spread = 11.27
Class 7: center = (12.35, -9.74), intra-class spread = 11.97
Class 8: center = (23.20, 21.68), intra-class spread = 8.81
Class 9: center = (-48.84, -52.73), intra-class spread = 43.36
```

شکل 14 - مرکز کلاس ها و واریانس درون کلاسی

```
Average inter-class distance: 63.84
Average intra-class spread: 20.50
```

شکل 15 - واریانس بین کلاسی و درون کلاسی بر اساس فیچر ها

زیربخش سوم: پیاده سازی ANGULAR MARGIN LOSS

الف) در روش های مبتنی بر فاصله زاویه ای، به جای استفاده از ضرب داخلی ساده هر دو بردار ویژگی و وزن را به سطح کره واحد (unit hypersphere) نرمال می کنیم. این کار باعث می شود که خروجی لایه fully connected مستقیماً برابر با کسینوس زاویه بین دو بردار باشد. این کار در تابع زیر در شبکه پیاده سازی شده انجام شده است:

```
def forward(self, x):
    x = self.features(x)
    embedding = self.embedding(x)

    embedding_norm = torch.nn.functional.normalize(embedding, p=2, dim=1)
    weight_norm = torch.nn.functional.normalize(self.fc.weight, p=2, dim=1)
    logits = torch.mm(embedding_norm, weight_norm.t())

    return logits, embedding_norm
```

شکل 16 - نرمالسازی $L2$

(ب) در این تابع، ابتدا یک ماتریس onehot از برچسبها ساخته می‌شود. سپس مقدار margin فقط از ستون‌های مربوط به کلاس صحیح کم می‌شود. به این ترتیب، logits کلاس صحیح به اندازه m کاهش می‌یابد و شبکه مجبور می‌شود ویژگی‌هایی تولید کند که زاویه کمتری با وزن کلاس صحیح داشته باشند. در نهایت یک ضریب scale برای بهبود همگرایی به logits اعمال شده و loss با استفاده از CrossEntropy محاسبه می‌شود.

```
def forward(self, logits, labels):
    batch_size = logits.size(0)

    one_hot = torch.zeros_like(logits)
    one_hot.scatter_(1, labels.unsqueeze(1), 1)

    cos_theta = logits
    cos_theta_m = cos_theta - self.margin * one_hot

    final_logits = self.scale * cos_theta_m
    loss = torch.nn.functional.cross_entropy(final_logits, labels)

    return loss
```

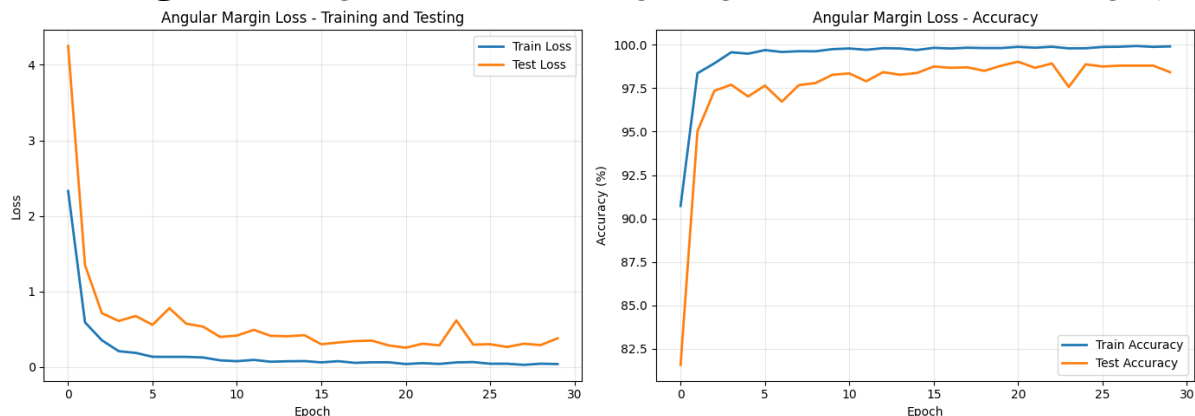
شکل 17 - پیاده سازی $angular\ margin\ loss$

(ج) با آموزش این مدل به دقت ۹۸ درصد روی داده های تست رسیدیم.

Epoch 5/30	Train Loss: 0.1880	Train Acc: 99.49%	Test Loss: 0.6757	Test Acc: 97.03%
Epoch 10/30	Train Loss: 0.0892	Train Acc: 99.75%	Test Loss: 0.3993	Test Acc: 98.28%
Epoch 15/30	Train Loss: 0.0793	Train Acc: 99.70%	Test Loss: 0.4221	Test Acc: 98.38%
Epoch 20/30	Train Loss: 0.0633	Train Acc: 99.81%	Test Loss: 0.2862	Test Acc: 98.80%
Epoch 25/30	Train Loss: 0.0664	Train Acc: 99.80%	Test Loss: 0.2974	Test Acc: 98.88%
Epoch 30/30	Train Loss: 0.0400	Train Acc: 99.91%	Test Loss: 0.3803	Test Acc: 98.42%

شکل 18 - نتیجه آموزش مدل $angular\ margin$

همچنین نمودار خطا و دقت داده ها در حین آموزش به شکل زیر است که نشان دهنده همگرایی است:



شکل 19 - نمودار همگرایی خطا و دقت روی داده های تست و ترین

دقت نهایی مدل روی داده های ترین به ۹۹/۹ و روی داده های تست به ۹۸/۵ رسید. همچنین تغییر اندازه batch تغییر محسوسی در این مثال ایجاد نمیکند ولی به طور کلی افزایش batch size باعث همگرایی سریعتر و کاهش آن باعث دقت بیشتر میشود

```
Training with batch_size = 32
Epoch 5/15 | Train Loss: 0.2771 | Train Acc: 99.00% | Test Loss: 0.7783 | Test Acc: 96.95%
Epoch 10/15 | Train Loss: 0.1392 | Train Acc: 99.55% | Test Loss: 0.4484 | Test Acc: 98.25%
Epoch 15/15 | Train Loss: 0.0910 | Train Acc: 99.71% | Test Loss: 0.3473 | Test Acc: 98.58%
Batch size 32 -> Final test accuracy: 98.58%

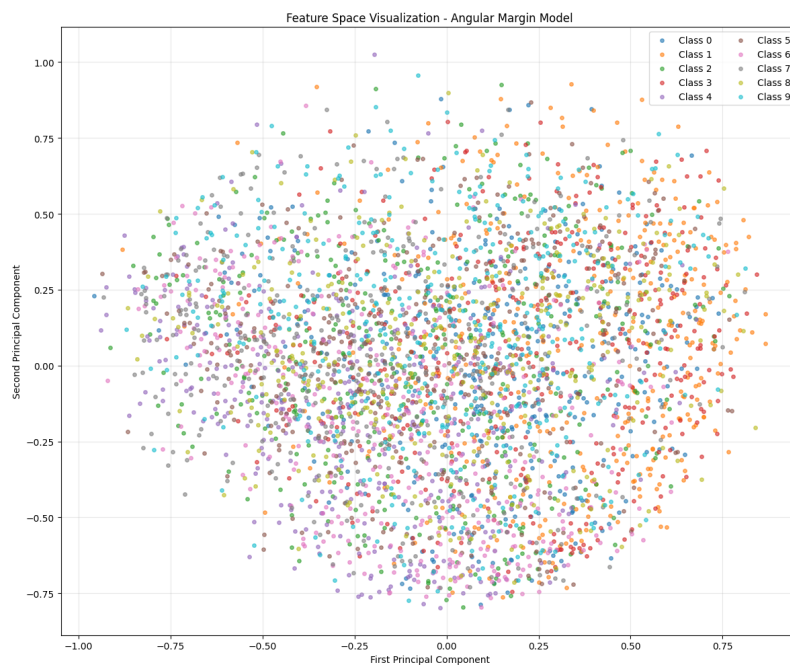
Training with batch_size = 64
Epoch 5/15 | Train Loss: 0.1872 | Train Acc: 99.46% | Test Loss: 1.1958 | Test Acc: 95.05%
Epoch 10/15 | Train Loss: 0.0871 | Train Acc: 99.76% | Test Loss: 0.4709 | Test Acc: 98.15%
Epoch 15/15 | Train Loss: 0.0651 | Train Acc: 99.83% | Test Loss: 0.3256 | Test Acc: 98.75%
Batch size 64 -> Final test accuracy: 98.75%

Training with batch_size = 128
Epoch 5/15 | Train Loss: 0.1103 | Train Acc: 99.86% | Test Loss: 0.4906 | Test Acc: 98.45%
Epoch 10/15 | Train Loss: 0.0794 | Train Acc: 99.88% | Test Loss: 0.5531 | Test Acc: 97.88%
Epoch 15/15 | Train Loss: 0.0561 | Train Acc: 99.90% | Test Loss: 0.5837 | Test Acc: 97.53%
Batch size 128 -> Final test accuracy: 97.53%

Effect of Batch Size on Angular Margin Loss
Batch size 32: 98.58%
Batch size 64: 98.75%
Batch size 128: 97.53%
```

شکل 20 - نمودار دقت مدل با batch size های مختلف

در این بخش کاهش بعد به کمک pca انجام شد ولی نتایج آن احتمالا دچار مشکل است و فاصله بین کلاسی کاهش و درون کلاسی مقادیر مناسبی را نشان نمیدهد.



شکل 21 - نمودار مقادیر دو فیچر با بیشترین واریانس در مدل

زیربخش چهارم: مقایسه نهایی، تحلیل و ارتباط با مقاله IAM

